

Diploma in Computer Science & Information Technology

SBTE, Bihar — Semester IV

Java Programming

Course Code: 2418401

Unit-wise Detailed Notes

UNIT 1: Basic Syntactical Constructs in Java

1.1 Java Features and Programming Environment

Java is a platform-independent, object-oriented, high-level programming language developed by James Gosling at Sun Microsystems in 1991 and officially released in 1995. It follows the principle "Write Once, Run Anywhere" (WORA) meaning compiled Java code (bytecode) can run on any platform with a JVM.

Key Features of Java

- **Simple:** Java syntax is based on C/C++ and removes complex features like pointers and multiple inheritance.
- **Platform Independent:** Java code is compiled into bytecode which runs on JVM on any OS.
- **Object-Oriented:** Everything in Java is treated as an object supporting encapsulation, inheritance, and polymorphism.
- **Robust:** Strong memory management, exception handling, and type checking make Java reliable.
- **Secure:** No explicit pointers; bytecode verified before execution by JVM.
- **Multithreaded:** Built-in support for multithreading allows concurrent execution.
- **Distributed:** Supports network programming with packages like java.net and java.rmi.
- **Portable:** Bytecode can be transferred over networks and executed on any JVM.

1.2 Object-Oriented Paradigm

Object-Oriented Programming (OOP) is a programming methodology that organizes software design around data (objects) rather than functions. Java is a fully object-oriented language.

Core OOP Concepts

- **Class:** A blueprint or template for creating objects. It defines attributes (fields) and behaviors (methods). Example: `class Car { int speed; void accelerate(){} }`
- **Object:** An instance of a class. Created using the new keyword. Example: `Car myCar = new Car();`
- **Data Abstraction:** Hiding complex implementation details and showing only essential features. Achieved using abstract classes and interfaces.
- **Encapsulation:** Wrapping data and methods into a single unit (class) and restricting direct access using access modifiers (private, protected, public).
- **Inheritance:** Mechanism where a child class acquires properties and behaviors of a parent class using the extends keyword. Promotes code reusability.
- **Polymorphism:** Ability of one method to take many forms. Achieved through method overloading (compile-time) and method overriding (runtime).

1.3 Java vs C and Java vs C++

Java vs C

- Java is object-oriented; C is procedural.
- Java has no pointers; C uses explicit pointer arithmetic.
- Java has automatic garbage collection; C requires manual memory management.
- Java is platform-independent; C is platform-dependent.

Java vs C++

- Java does not support multiple inheritance via classes (uses interfaces); C++ supports it.
- Java has no destructor concept; uses garbage collector.
- Java has no operator overloading except for the + operator (string concatenation).
- Java programs run on JVM; C++ compiles to native machine code.

1.4 Java Development Environment

JDK (Java Development Kit)

JDK is a complete development kit that includes JRE (Java Runtime Environment) + development tools such as javac (compiler), java (interpreter), javadoc, jar, and debugger.

JVM (Java Virtual Machine)

JVM is an abstract machine that provides the runtime environment to execute Java bytecode. It performs: Loading of .class files, Bytecode verification, Execution (interpretation/JIT compilation), Garbage collection.

Bytecode

When Java source code (.java) is compiled by javac, it produces an intermediate code called bytecode (.class file). This bytecode is platform-independent and is interpreted by the JVM on any operating system.

1.5 Java API Packages

- **java.lang:** Fundamental classes like String, Math, Object, System (auto-imported).
- **java.util:** Utility classes like ArrayList, HashMap, Scanner, Date.
- **java.io:** Input/Output classes for file and stream handling.
- **java.awt:** Abstract Window Toolkit for GUI components.
- **java.applet:** For creating applets that run in browsers.
- **java.net:** Classes for network programming (sockets, URL handling).

1.6 Java Program Structure

A Java program consists of several sections: Documentation section (comments), Package statement (optional), Import statements, Interface declarations, Class definitions, and the main method.

Example structure:

```
public class HelloWorld { public static void main(String[] args) { System.out.println("Hello, World!"); } }
```

1.7 Java Tokens

Tokens are the smallest individual units of a Java program. Java recognizes five types of tokens:

- **Keywords:** Reserved words with predefined meaning — class, public, static, void, int, if, for, while, return, new, etc.
- **Identifiers:** Names given to variables, methods, classes. Must start with a letter, underscore () or dollar sign (\$). Case-sensitive.
- **Literals:** Constant values — integer (10), float (3.14f), char ('A'), string ("Hello"), boolean (true/false).
- **Operators:** Symbols performing operations — arithmetic (+, -, *, /, %), relational (==, !=, <, >), logical (&&, ||, !), bitwise (&, ^, ~), etc.
- **Separators:** Symbols used to separate tokens — { } () [] ; , .

1.8 Data Types in Java

Primitive Data Types

- **byte:** 8-bit signed integer. Range: -128 to 127.
- **short:** 16-bit signed integer. Range: -32,768 to 32,767.
- **int:** 32-bit signed integer. Range: -2^{31} to $2^{31}-1$.
- **long:** 64-bit signed integer. Suffix L.
- **float:** 32-bit IEEE 754 floating-point. Suffix f.
- **double:** 64-bit IEEE 754 floating-point (default decimal type).
- **char:** 16-bit Unicode character. Range: '\u0000' to '\uffff'.
- **boolean:** Represents true or false.

Non-Primitive Data Types

Also called Reference types — String, Arrays, Classes, Interfaces. They store references (memory addresses) to objects.

1.9 Operators in Java

- **Arithmetic:** +, -, *, /, % (modulus)
- **Relational:** ==, !=, <, >, <=, >= — return boolean values
- **Logical:** && (AND), || (OR), ! (NOT)
- **Assignment:** =, +=, -=, *=, /=, %=
- **Increment/Decrement:** ++ (pre/post increment), -- (pre/post decrement)
- **Ternary:** condition ? expr1 : expr2 — shorthand for if-else
- **Bitwise:** & (AND), | (OR), ^ (XOR), ~ (complement), << (left shift), >> (right shift)

1.10 Control Statements

Decision Statements

- **if statement:** Executes block if condition is true.
- **if-else:** Executes one of two blocks based on condition.
- **nested if-else:** if-else inside another if-else.
- **if-else-if ladder:** Tests multiple conditions in sequence.
- **switch:** Multi-way branch based on value of an integer or char expression. Uses case, break, and default keywords.

Loop Statements

- **while loop:** Entry-controlled; checks condition before each iteration.
- **do-while loop:** Exit-controlled; executes body at least once, then checks condition.
- **for loop:** Most compact form with initialization, condition, and update in one line.
- **for-each loop:** Enhanced for loop used to iterate over arrays or collections.

Jump Statements

- **break:** Exits the current loop or switch statement immediately.
 - **continue:** Skips the current iteration and moves to the next.
 - **return:** Exits the current method and optionally returns a value.
-

UNIT 2: Derived Syntactical Constructs in Java

2.1 Constructors

A constructor is a special method that is automatically called when an object of a class is created. It initializes the object. Constructor has the same name as the class and has no return type (not even void).

Types of Constructors

- **Default Constructor:** Provided automatically by Java if no constructor is defined. Initializes instance variables to default values (0, null, false).
- **Non-Parameterized Constructor:** A constructor written by the programmer with no parameters. Used to set specific default values.
- **Parameterized Constructor:** A constructor that accepts arguments to initialize object attributes with custom values. Example: `Student(int id, String name){ this.id=id; this.name=name; }`

this Keyword

The `this` keyword refers to the current object instance. It is used to: (1) Distinguish instance variables from local variables when they have the same name, (2) Call another constructor in the same class (constructor chaining), (3) Pass the current object as argument to a method.

2.2 Variable-Length Arguments (Varargs)

Varargs allow a method to accept a variable number of arguments of the same type. Syntax: `void method(int... nums)`. Internally treated as an array. Example: `void sum(int... n){ int s=0; for(int x:n) s+=x; }`

2.3 Visibility Control (Access Modifiers)

Access modifiers control the accessibility of classes, methods, and variables:

- **public:** Accessible from any class in any package.
- **private:** Accessible only within the same class. Provides strongest encapsulation.
- **protected:** Accessible within the same package and by subclasses in other packages.
- **default (no modifier):** Accessible only within the same package (package-private).

2.4 Arrays in Java

An array is a collection of elements of the same data type stored in contiguous memory locations. Arrays are objects in Java.

Types of Arrays

- **Single-Dimensional Array:** Linear collection. Declaration: `int[] arr`; Creation: `arr = new int[5]`; Initialization: `int[] arr = {10,20,30,40,50}`;
- **Multi-Dimensional Array:** Array of arrays. 2D array: `int[][] matrix = new int[3][4]`; Represents rows and columns.

Array length can be found using `arr.length`. Arrays are zero-indexed (first element at index 0).

2.5 Strings in Java

String Class

String objects are immutable in Java — once created, their value cannot be changed. Strings can be created as: `String s = "Hello";` or `String s = new String("Hello");`

Important String methods: `length()`, `charAt(i)`, `substring(start,end)`, `equals()`, `equalsIgnoreCase()`, `toUpperCase()`, `toLowerCase()`, `trim()`, `indexOf()`, `replace()`, `concat()`, `compareTo()`

StringBuffer Class

StringBuffer is mutable — its content can be changed after creation. Thread-safe (synchronized). Important methods: `append()`, `insert()`, `delete()`, `replace()`, `reverse()`, `length()`. Used when frequent string modification is needed.

Note: StringBuilder is similar to StringBuffer but not synchronized (faster for single-threaded use).

UNIT 3: Inheritance, Interface and Package

3.1 Concept of Inheritance

Inheritance is the mechanism by which a new class (child/subclass/derived class) acquires the properties and methods of an existing class (parent/superclass/base class). It promotes code reusability and establishes an IS-A relationship. Java uses the `extends` keyword for inheritance.

The `super` keyword is used to: Call the parent class constructor (`super()`), Call the parent class method or access parent class variable.

3.2 Types of Inheritance

- **Single Inheritance:** A class inherits from only one parent class. Example: class Dog extends Animal
`{}`
- **Multilevel Inheritance:** A class inherits from a class which itself inherits from another. Example: A extends B, B extends C. Forms a chain.
- **Hierarchical Inheritance:** Multiple classes inherit from a single parent class. Example: Dog and Cat both extend Animal.
- **Multiple Inheritance:** Java does NOT support multiple inheritance through classes to avoid the "Diamond Problem." It is achieved through interfaces.

3.3 Method Overloading and Overriding

Method Overloading (Compile-time Polymorphism)

Multiple methods in the same class with the same name but different parameter lists (different number, type, or order of parameters). Return type alone cannot distinguish overloaded methods.

Method Overriding (Runtime Polymorphism)

A subclass provides its own specific implementation of a method that is already defined in the parent class. The method must have the same name, same parameters, and same return type. The `@Override` annotation is used for clarity.

3.4 Interface

An interface in Java is a fully abstract type — it defines what a class must do but not how. All methods in an interface are by default public and abstract. All fields are public, static, and final. A class implements an

interface using the implements keyword.

Key Points

- A class can implement multiple interfaces (achieves multiple inheritance).
- Interfaces can extend other interfaces using extends.
- From Java 8 onwards, interfaces can have default and static methods with implementation.

3.5 Packages

A package is a namespace that organizes related classes and interfaces into a directory-like structure. Packages prevent naming conflicts and provide access control.

Types of Packages

- **Built-in Packages:** Provided by Java API — java.lang, java.util, java.io, java.awt, java.net, etc.
- **User-defined Packages:** Created by the programmer using the package keyword at the top of the source file.

Using Packages

- Import entire package: `import java.util.*;`
 - Import specific class: `import java.util.Scanner;`
 - Fully qualified name (no import needed): `java.util.Scanner sc = new java.util.Scanner(System.in);`
-

UNIT 4: Errors & Exception Handling, Multithreading

4.1 Types of Errors

- **Compile-time Errors (Syntax Errors):** Detected by the Java compiler before execution. Examples: missing semicolon, undeclared variable, mismatched braces, wrong data type usage.
- **Runtime Errors:** Occur during program execution. Examples: dividing by zero, accessing null reference, array index out of bounds. These cause program termination unless handled.
- **Logical Errors:** Program runs without crashing but produces wrong output due to incorrect logic. Hardest to detect — requires debugging.

4.2 Exception Handling

An exception is an abnormal event that disrupts the normal flow of a program at runtime. Java provides a robust exception-handling mechanism using: try, catch, finally, throw, throws.

try-catch Block

The try block contains code that might throw an exception. The catch block handles the exception. Syntax: `try { risky code } catch(ExceptionType e) { handler code }`

Multiple catch Blocks

A try block can have multiple catch blocks for different exception types. The most specific exception should be caught first (subclass before superclass).

finally Block

The finally block always executes whether an exception is thrown or not. Used for cleanup tasks like closing files or database connections. Syntax: try{...} catch(Exception e){...} finally{ always runs }

Nested try Statements

A try block can be nested inside another try block. If the inner try does not have a matching catch, the outer try's catch is searched.

throw and throws

- **throw:** Used to explicitly throw an exception from a method. Example: throw new ArithmeticException("Division by zero");
- **throws:** Declares that a method may throw certain checked exceptions. Placed in the method signature. Example: void readFile() throws IOException {}

Built-in Exceptions

- ArithmeticException — integer division by zero
- NullPointerException — accessing a null object reference
- ArrayIndexOutOfBoundsException — invalid array index
- ClassCastException — invalid type casting
- NumberFormatException — invalid numeric string conversion
- IOException — general I/O operation failures

4.3 Multithreading

Multithreading is the ability to execute multiple threads concurrently within a single program. A thread is a lightweight subprocess — the smallest unit of execution. Multithreading improves CPU utilization and program responsiveness.

Creating Threads — Two Ways

- **Method 1 — Extending Thread class:** class MyThread extends Thread { public void run(){ // task } }
Then: MyThread t = new MyThread(); t.start();
- **Method 2 — Implementing Runnable interface:** class MyTask implements Runnable { public void run(){ // task } }
Then: Thread t = new Thread(new MyTask()); t.start();

4.4 Life Cycle of a Thread

- **New State:** Thread object created but start() not yet called.
- **Runnable State:** Thread ready to run; waiting for CPU time from scheduler. start() has been called.
- **Running State:** Thread is actively executing its run() method.
- **Blocked/Waiting State:** Thread temporarily paused due to sleep(), wait(), or waiting for I/O.
- **Dead State:** Thread has completed execution of run() method or was terminated. Cannot be restarted.

4.5 Thread Methods

- **sleep(ms):** Pauses thread execution for specified milliseconds.
- **wait():** Causes thread to wait until notified (used in synchronization).
- **notify():** Wakes up a single thread that is waiting.
- **notifyAll():** Wakes up all threads that are waiting.
- **suspend() / resume() / stop():** Deprecated methods — not recommended for use.

- **setPriority(n)**: Sets thread priority (1 = MIN_PRIORITY, 10 = MAX_PRIORITY, 5 = NORM_PRIORITY).
-

UNIT 5: Managing Input/Output/Files in Java

5.1 Concept of Streams

In Java, I/O operations are performed through streams. A stream is a sequence of data flowing from a source (input) to a destination (output). Java I/O is defined in the java.io package. Streams are of two types based on data type: Byte Streams and Character Streams.

5.2 Byte Stream Classes

Byte streams handle I/O of raw binary data (8-bit bytes). Useful for image, audio, and video file handling.

- **InputStream (abstract)**: Base class for all byte input streams. Key method: read()
- **OutputStream (abstract)**: Base class for all byte output streams. Key method: write()
- **FileInputStream**: Reads bytes from a file.
- **FileOutputStream**: Writes bytes to a file.
- **BufferedInputStream / BufferedOutputStream**: Add buffering for efficient I/O.
- **DataInputStream / DataOutputStream**: Read/write primitive data types (int, float, boolean) from/to streams.

5.3 Character Stream Classes

Character streams handle I/O of characters (16-bit Unicode). Suitable for text file handling.

- **Reader (abstract)**: Base class for character input. Key method: read()
- **Writer (abstract)**: Base class for character output. Key method: write()
- **FileReader**: Reads characters from a file.
- **FileWriter**: Writes characters to a file.
- **BufferedReader**: Adds buffering; provides readLine() for reading complete lines.
- **BufferedWriter**: Adds buffering; provides newLine() for writing line separators.
- **PrintWriter**: Writes formatted text; provides println(), print(), printf().

5.4 File Class

The File class in java.io represents file and directory pathnames abstractly. It does not read/write file contents but provides metadata and file system operations.

Common File Methods

- exists() — checks if file/directory exists
- createNewFile() — creates a new empty file
- delete() — deletes file or directory
- mkdir() / mkdirs() — creates directory/directories
- getName(), getPath(), getAbsolutePath() — file name/path info
- length() — file size in bytes

- `isFile()`, `isDirectory()` — type checking
- `list()` — returns array of filenames in a directory

5.5 Reading and Writing Files

Reading bytes: Use `FileInputStream` wrapped in `BufferedInputStream` for efficiency. Always close streams in a finally block or use `try-with-resources` (Java 7+).

Writing bytes: Use `FileOutputStream`. Pass `true` as second argument for append mode: `new FileOutputStream("file.txt", true)`

Reading text: `BufferedReader br = new BufferedReader(new FileReader("file.txt")); String line; while((line=br.readLine())!=null){ System.out.println(line); } br.close();`

Note: Always handle `IOException` when working with file streams. Use `try-with-resources` for automatic resource management.
